

DISEÑO DE APIS

REST & HATEOAS

Principios aplicados sobre la **Pet Store API**

Recursos como sustantivos

Hipermedia

HTTP semántico

REST — EL PROBLEMA

La Pet Store original — ¿qué tienen en común estas URLs?

GET /pet/findByStatus?status=available

GET /pet/findByTags?tags=indoor

POST /pet/{petId}/uploadImage

GET /user/login

POST /user/createWithList

Todas son válidas técnicamente. ¿Cuál es el problema?

Las URLs identifican **recursos**, no acciones

✗ Verbos en la URL

GET /pet/findByStatus?status=available

GET /pet/findByTags?tags=indoor

POST /pet/{id}/uploadImage
— sin GET de imágenes —

GET /user/login

POST /user/createWithList

✓ Sustantivos — recursos

GET /pets?status=available

GET /pets?tags=indoor

GET /pets/{id}/**images**
POST /pets/{id}/**images**
GET /pets/{id}/**images**/**{imageId}**

~~— eliminado —~~

~~— eliminado —~~

Regla: el verbo lo aporta el método HTTP — la URL solo nombra el recurso.

Sin convención, cada endpoint inventa su patrón

GET /Pet

singular

GET /pet/id/10

id como segmento literal

GET /pet/getImages?petId=10

verbo + query param

POST /pet/uploadImage/10

verbo en path

GET /getOrder?id=42

patrón diferente

GET /User/theUser

PascalCase

El cliente necesita documentación para cada URL — no puede inferir ninguna.

Colecciones en plural, jerarquía clara

✗ Sin convención — caos acumulado

GET /Pet	singular
GET /pet/id/10	id como segmento literal
GET /pet/getImages?petId=10	verbo + query param
POST /pet/uploadImage/10	verbo en path
GET /getOrder?id=42	patrón diferente
GET /User/theUser	singular, PascalCase

Cada endpoint sigue su propio criterio. El cliente necesita documentación para cada URL.

✓ Plural + jerarquía — patrón predecible

GET /pets	colección
GET /pets/10	recurso individual
GET /pets/10/images	sub-colección
POST /pets/10/images	→ 201 + Location
GET /store/orders/42	namespace del dominio
GET /users/theUser	ID semántico
/recurso/{id}/sub-recurso — la jerarquía expresa pertenencia. Si conocés el patrón, podés inferir cualquier URL.	

REST — EL PROBLEMA

Toda acción cabe en un POST — ¿o no?

```
POST /api?action=createPet
POST /api?action=getPet&id=10
POST /api?action=updatePet&id=10
POST /api?action=deletePet&id=10
```

El servidor entiende cada acción. Funciona. ¿Qué se pierde?

Cada método HTTP tiene una **semántica precisa**

✗ El antipatrón: todo con POST

```
POST /api?action=createPet
POST /api?action=getPet&id=10
POST /api?action=updatePet&id=10
POST /api?action=deletePet&id=10
```

HTTP se convierte en un túnel. Se pierde caché, idempotencia, semántica — todo lo que el protocolo ya resuelve gratis.

✓ Un método para cada intención

GET Lectura. Safe & idempotente. Cacheable.

POST Crea. No idempotente. Retorna **201** + Location.

PUT Reemplaza el recurso completo. Idempotente.

PATCH Actualización parcial. Solo los campos que cambian.

DELETE Elimina. Retorna **204** sin cuerpo.

EN LA PRÁCTICA

```
GET /pets/10
GET /pets?status=available
```

```
POST /pets
→ 201 Created
   Location: /pets/11
```

```
PUT /pets/10
{ "name": "Rex", "status": "available", ... }
```

```
PATCH /pets/10
{ "status": "sold" }
```

```
DELETE /pets/10
→ 204 No Content
```

El método comunica la **intención**. La URL identifica el **recurso**. Ninguno invade el rol del otro.

REST — EL PROBLEMA

Siempre **200 OK** — ¿qué tiene de malo?

```
POST /pets → 200 OK  
{ "success": true, "id": 11 }
```

```
GET /pets/999 → 200 OK  
{ "success": false, "error": "not found" }
```

```
DELETE /pets/10 → 200 OK  
{ "deleted": true }
```

El servidor responde. El cliente puede leer el body. ¿Cuál es el problema?

Códigos de estado semánticos

✗ El antipatrón: siempre 200

```
POST /pets → 200 OK
{ "success": true, "id": 11 }

GET /pets/999 → 200 OK
{ "success": false, "error": "not found" }

DELETE /pets/10 → 200 OK
{ "deleted": true }
```

El cliente tiene que parsear el body para saber si la operación falló. El código HTTP deja de comunicar. Los proxies y cachés se confunden.

✓ El código HTTP es el mensaje

```
POST /pets → 201 Created
Location: /pets/11
{ "id": 11, "name": "Fluffy", ... }

GET /pets/10 → 200 OK
{ "id": 10, "name": "Rex", ... }

GET /pets/999 → 404 Not Found
{ "code": "PET_NOT_FOUND", "message": "..." }

DELETE /pets/10 → 204 No Content
(sin body)
```

El código HTTP es suficiente para saber si algo salió bien. El body solo existe cuando hay contenido que aportar.

HATEOAS

Hypermedia as the Engine of **Application State**

El servidor le dice al cliente **qué puede hacer a continuación**. El cliente nunca hardcodea URLs.

Sin HATEOAS

El cliente conoce de antemano cada URL y cada operación disponible. Si la API cambia, el cliente se rompe.

Con HATEOAS

El cliente sigue los links que el servidor incluye en cada respuesta. La API es autodescriptiva y navegable.

El cliente conoce la API — la construyó así

```
const pet = await fetch("/pets/10");

// El cliente sabe cómo son las URLs
const imageUrl = `/pets/${pet.id}/images`;
const selfUrl   = `/pets/${pet.id}`;

// Si el equipo de API mueve las imágenes a /media/...
const imageUrl = `/media/pets/${pet.id}`; ← cambio manual en el cliente
```

Mientras la API no cambie, funciona perfecto. ¿Qué pasa cuando cambia?

Cada recurso expone sus relaciones navegables

✗ Sin HATEOAS — el cliente sabe de antemano

```
// El cliente construye URLs por su cuenta
const pet = await fetch("/pets/10");

const imageUrl = `/pets/${pet.id}/images`;
const selfUrl = `/pets/${pet.id}`;
```

Si el equipo de API mueve las imágenes a `/media/pets/10`, el cliente se rompe. El contrato estaba en la cabeza del desarrollador, no en la respuesta.

✓ Con HATEOAS — el servidor describe las relaciones

```
GET /pets/10 → 200 OK

{
  "id": 10,
  "name": "Rex",
  "status": "available",

  "_links": {
    "self": { "href": "/pets/10" },
    "images": { "href": "/pets/10/images" }
  }
}
```

El cliente solo sigue `_links.images.href`. Si la URL cambia, el servidor actualiza el link — el cliente no se entera ni le importa.

OPTIONS /pets/10 — para descubrir qué métodos HTTP admite el recurso. Responde `Allow: GET, PUT, PATCH, DELETE`.

Paginar en el body — el approach más intuitivo

GET /pets?page=2 → 200 OK

```
{
  "items": [ /* pets */ ],
  "pagination": {
    "page": 2,
    "pageSize": 20,
    "total": 98,
    "next": "/pets?page=3"
  }
}
```

Toda la info está ahí. ¿Por qué sería un problema?

Paginación en el header **Link**, no en el body

✗ El envelope artificial

GET /pets?page=2 → 200 OK

```
{
  "items": [ /* pets */ ],
  "pagination": {
    "page": 2,
    "total": 98,
    "next": "/pets?page=3"
  }
}
```

GET /pets debería devolver pets — no un objeto inventado que los envuelve. La paginación es metadata de transporte, no del dominio.

Body limpio. GET /pets devuelve exactamente lo que dice: un array de pets. Sin wrappers artificiales.

Estándar HTTP. RFC 8288 define el header Link exactamente para esto. GitHub API lo usa así desde hace años.

Cuando rel="next" **no aparece**, no hay más páginas. Sin parsear contadores ni calcular offsets.

```
// El cliente itera siguiendo links
while (response.headers.get("Link")?.includes('rel="next"')) {
  response = await fetchNext(response);
}
```

✓ Header Link (RFC 8288)

GET /pets?page=2 → 200 OK

Link: </pets?page=1>; rel="prev",
 </pets?page=3>; rel="next",
 </pets?page=5>; rel="last"

X-Total-Count: 98

[/* array limpio de pets */]

El cliente **sigue links**, no hardcodea URLs

GET /pets

→ `[0]._links.self.href = "/pets/10"`
← sigue el link del primer resultado

GET /pets/10

→ `_links.images = { href: "/pets/10/images" }`
← navega a las imágenes

GET /pets/10/images

→ `[0]._links.self.href = "/pets/10/images/7"` ← imagen individual
→ `_links.pet.href = "/pets/10"` ← link al padre

POST /store/orders

201 Location: /store/orders/99
→ `_links.pet.href = "/pets/10"`
← link cruzado al recurso relacionado

El cliente arranca desde /pets y navega toda la API siguiendo links. Si mañana /pets/10/images cambia a /media/pets/10, el cliente no se rompe — actualiza el servidor, no el cliente.

AUTH — EL PROBLEMA

La autenticación como acción — el endpoint /login

POST /login → 200 + token

POST /user/login → 200 + token

POST /auth/token → 200 + access_token

GET /user/logout → 200

Una URL dedicada para "entrar". Funciona — ¿qué principio viola?

No existe `/login` — la auth viaja en el `header`

✗ Verbos de sesión en la URL

```
POST /login
```

```
GET /user/login
```

```
POST /user/logout
```

Estos endpoints mezclan el *transporte* de credenciales con la semántica de recursos. No representan ningún sustantivo del dominio.

✓ HTTP Basic — estándar del protocolo

Las credenciales viajan en el header `Authorization`, definido por RFC 7617. Cualquier endpoint las acepta — no hay una URL especial de login.

```
GET /pets
```

```
Authorization: Basic dXNlcjpwYXNz
               ↑ base64(username:password)
```

```
200 OK
```

```
X-Access-Token: eyJ...
```

```
X-Refresh-Token: dGt...
```

```
[ /* pets — body limpio, solo el recurso */ ]
```

Los tokens son metadata de transporte — van en headers, igual que `Location` o `Link`. El body solo contiene el recurso pedido.

Flujo de access & refresh token

GET /pets ● Basic Auth	Authorization: Basic dXNlcjpwYXNz → 200 OK X-Access-Token: eyJ... X-Refresh-Token: dGt... ← tokens en headers
GET /pets/10 ● Bearer access	Authorization: Bearer eyJ... → 200 OK ← requests normales mientras el access token esté vigente
DELETE /pets/10 ● Sin permiso	Authorization: Bearer eyJ... ← token válido, pero el usuario no es admin → 403 Forbidden { "code": "INSUFFICIENT_PERMISSIONS" } ← no refrescar — el problema no es el token
GET /store/orders ● Token vencido	Authorization: Bearer eyJ... ← mismo token, pero expiró → 401 Unauthorized { "code": "TOKEN_EXPIRED" } ← este sí se resuelve con el refresh token
GET /store/orders ● Bearer refresh token	Authorization: Bearer dGt... ← Bearer, pero con el refresh token → 200 OK X-Access-Token: eyJ... X-Refresh-Token: dGt... ← rota ambos tokens

401 Unauthorized — el token venció o es inválido. El cliente debe refrescar con el Bearer refresh token.	403 Forbidden — el token es válido pero el usuario no tiene permisos. Refrescar no sirve — es un problema de autorización.
---	---

RESUMEN

Lo que cambió en la Pet Store API

REST

- ✓ URLs solo con sustantivos en plural
- ✓ Verbos eliminados (`findBy*`, `upload*`, `createWith*`)
- ✓ Filtros como query params en la colección
- ✓ ID del recurso siempre en la URL
- ✓ PUT reemplaza, PATCH actualiza parcialmente
- ✓ POST → 201 + Location
- ✓ DELETE → 204 sin cuerpo

HATEOAS

- ✓ `_links` en todos los recursos y colecciones
- ✓ Paginación en header `Link` (RFC 8288), body limpio
- ✓ Relaciones navegables declaradas en la respuesta
- ✓ Recursos relacionados enlazados (pet desde order)
- ✓ `photoUrls` reemplazado por sub-recurso `/pets/{id}/images`
- ✓ Imágenes navegables: GET `/pets/{id}/images` → GET `/pets/{id}/images/{imageId}` (image/*)

Autenticación

- ✓ Sin `/login` — credenciales en header `Authorization`
- ✓ HTTP Basic en el primer request → `X-Access-Token` + `X-Refresh-Token` en headers
- ✓ Requests siguientes con Bearer `<accessToken>`
- ✓ **401** por token vencido → reintento con `refreshToken` → nuevo par